



TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

ARTHUR GUIMARÃES MIRANDA PRATES

DANIEL DA SILVA OLIVEIRA

FELIPE ANDRADE DOS SANTOS CARVALHO

MARCOS HENRIQUE BRITO SOUZA

**UMA ANÁLISE TÉCNICA DE MÉTODOS DE ESCALONAMENTO DE
PROCESSOS E SUA SIMULAÇÃO EM LINGUAGEM C**

GUANAMBI

2025

ARTHUR GUIMARÃES MIRANDA PRATES
DANIEL DA SILVA OLIVEIRA
FELIPE ANDRADE DOS SANTOS CARVALHO
MARCOS HENRIQUE BRITO SOUZA

**UMA ANÁLISE TÉCNICA DE MÉTODOS DE ESCALONAMENTO DE
PROCESSOS E SUA SIMULAÇÃO EM LINGUAGEM C**

Trabalho apresentado ao Instituto Federal
de Educação Ciência e Tecnologia
Baiano, como requisito de avaliação
parcial da disciplina Sistemas
Operacionais e, por conseguinte, a
obtenção dos seus créditos.

Profº. Cleiton Brito

GUANAMBI

2025

SUMÁRIO

1	SJF (SHORTEST JOB FIRST).....	4
2	ESCALONAMENTO ROUND ROBIN	5
2.1	VANTAGENS.....	5
2.2	DESVANTAGENS	5
2.3	APLICAÇÕES PRÁTICAS	5
3	FCFS (FIRST COME, FIRST SERVERD)	6
3.1	VANTAGENS.....	6
3.2	DESVANTAGENS	6
3.3	APLICAÇÕES PRÁTICAS	6
4	PRIORITY (PRIORITY SCHEDULING)	7
4.1	VANTAGENS.....	7
4.2	DESVANTAGENS	7
4.3	APLICAÇÕES PRÁTICAS	7
	REFERÊNCIAS.....	8

1 SJF (SHORTEST JOB FIRST)

O SJF “Shortest Job First” é um algoritmo de escalonamento de processos em que a CPU sempre escolhe o processo com o menor tempo de execução primeiro, para definir o tempo de execução usa-se o tempo de burst, que se refere a quanto tempo será utilizado da CPU. Verifica todos os processos e analisa seu tempo de burst, selecionando o que tem o menor valor e utilizando ele como primeiro elemento, seguindo esse sistema o último elemento será o que tem o maior tempo de burst. O SJF pode ser de dois tipos, Não-preemptivo, depois que a CPU começa um processo, vai até o fim ou Preemptivo também chamado de SRTF “Shortest Remaining Time First”, Que quando o processo menor chegar enquanto outro está rodando, ele interrompe o atual, e inicia o menor.

Processo	Tempo de Burst
P1	6
P2	4
P3	8
P4	2

Ordem pelo SJF → P4 (2) → P2 (4) → P1 (6) → P3 (8)

1.1 VANTAGENS

Diminui o tempo médio de espera dos processos, aumenta o aproveitamento da CPU já que os processos menores são executados primeiro e é ótimo teoricamente, para não-preemptivo, já que ele entrega menor tempo médio de espera possível.

1.2 DESVANTAGENS

Difícil saber o tempo de burst real de cada processo, pois ele trabalha com estimativas, pode causar inanição “starvation”, quando processos longos ficam esperando sempre, sendo também umas das mais difíceis de se implementar.

1.3 APLICAÇÕES PRÁTICAS

Processamento em lote, otimização do tempo médio de espera, favorecimento de processos i/o-bound (muitos processos de entrada e saída) são exemplos de uso do SJF.

2 ESCALONAMENTO ROUND ROBIN

Round Robin é uma técnica de gerenciamento de processos que distribui o tempo de CPU de forma circular e equitativa entre todos os processos prontos para execução. Cada processo recebe uma fatia de tempo denominada quantum, e ao final deste período, caso não tenha concluído sua execução, retorna ao final da fila de processos, permitindo que o próximo processo seja executado (TANENBAUM).

Processo	Tempo	Ação	Fila
P1	3ms	Início (quantum=2ms)	P1, P2,
P1	1ms	Fim do quantum, volta para o fim da fila	P2, P1
P2	2ms	Início (quantum=2ms)	P2, P1
P2	0ms	Fim do quantum, foi executado	P1
P1	2ms	Início do 2º ciclo (quantum=2ms)	P1
P1	0ms	Fim do quantum, foi executado	-

2.1 VANTAGENS

Equidade: Todos os processos recebem tempo de CPU de forma justa e previsível. Ausência de starvation: Nenhum processo fica indefinidamente sem execução. Simplicidade: Implementação invariável e direta. Tempo de resposta: Adequado para sistemas interativos e compartilhados

2.2 DESVANTAGENS

Overhead de contexto: Trocas frequentes aumentam o custo computacional Quantum crítico: Valores inadequados podem degradar o desempenho significativamente. Ineficiência para I/O: Processos que aguardam operações de entrada/saída desperdiçam quantum. Turnaround médio: Pode ser superior a algoritmos que priorizam processos mais curtos

2.3 APLICAÇÕES PRÁTICAS

O algoritmo Round Robin é utilizado em sistemas operacionais de tempo compartilhado, como Unix, Linux e Windows, para gerenciar processos em ambientes multiusuário. Sendo eficaz em servidores web que atendem múltiplas requisições simultâneas, sistemas de time-sharing acadêmicos onde vários usuários compartilham recursos computacionais, e em sistemas embarcados de tempo real.

3 FCFS (FIRST COME, FIRST SERVERD)

O FCFS é o algoritmo de escalonamento mais simples existente, funcionando como uma fila comum: o primeiro processo que chega é o primeiro a ser executado. Não há prioridades ou interrupções – cada processo usa a CPU do início ao fim antes de liberar para o próximo. Quando um processo entra na fila de prontos, ele espera sua vez baseado apenas na ordem de chegada. Como é não-preemptivo, a CPU não pode ser retirada de um processo em execução, mesmo que chegue outro mais rápido. Essa característica torna o algoritmo previsível, mas pode gerar esperas longas.

Processo	Tempo de Burst	Tempo de Burst
P1	0	8
P2	1	4
P3	2	2
P4	3	1

Ordem de execução: P1 (0-8) → P2 (8-12) → P3 (12-14) → P4 (14-15)

No exemplo, P4 precisa de apenas 1 unidade de tempo, mas espera 11 unidades porque chegou por último. Esse é o principal problema do FCFS.

3.1 VANTAGENS

Implementação trivial usando apenas uma fila simples. Todos os processos são eventualmente executados, evitando starvation. Não existe overhead de troca de contexto entre processos.

3.2 DESVANTAGENS

Sofre do efeito comboio, onde processos curtos ficam bloqueados por processos longos à frente na fila. O tempo médio de espera tende a ser alto. Inadequado para sistemas que exigem interatividade rápida.

3.3 APLICAÇÕES PRÁTICAS

Sistemas batch simples, impressoras em rede, processamento de requisições onde a ordem de chegada deve ser respeitada, e ambientes onde a justiça temporal é mais importante que eficiência.

4 PRIORITY (PRIORITY SCHEDULING)

O escalonamento Priority Scheduling é uma técnica em que cada processo recebe um valor de prioridade, e o sistema sempre seleciona para execução aquele que possui a prioridade mais alta. A prioridade pode ser definida pelo usuário, sistema ou critérios internos, como tempo de CPU, uso de memória ou importância da tarefa. Existem duas variações do método: preemptiva e não preemptiva. Na forma preemptiva, se um processo de maior prioridade chega enquanto outro está em execução, o sistema interrompe o processo atual e concede a CPU ao processo mais prioritário. Já na forma não preemptiva, o processo em execução somente libera a CPU quando seu tempo de execução termina.

Processos	Brust Time	Prioridade	Ordem de Execução
P1	5	2	2º
P2	1	1	1º
P3	10	3	3º
P4	1	5	4º

4.1 VANTAGENS

O escalonamento por prioridade apresenta como vantagem principal a capacidade de executar primeiro os processos mais importantes. Além disso, ele é flexível, permitindo ajustar prioridades conforme a necessidade, o que pode melhorar o desempenho em cenários específicos.

4.2 DESVANTAGENS

Por outro lado, o algoritmo pode causar starvation, já que processos com baixa prioridade podem esperar indefinidamente. Também depende muito da definição correta das prioridades, podendo gerar injustiças ou má distribuição dos recursos. Em sua forma preemptiva, pode aumentar o número de trocas de contexto, reduzindo a eficiência do sistema.

4.3 APLICAÇÕES PRÁTICAS

Gerenciamento de tráfego de rede, algoritmos de busca em grafos, atendimento em serviços de saúde/clínicas, inteligência artificial (busca heurística) são exemplos que necessitam de algoritmos de escalonamento como o priority scheduling.

REFERÊNCIAS

TANENBAUM, A. S.; BOS, H. Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson, 2016. Disponível em:
<https://plataforma.bvirtual.com.br/Leitor/Publicacao/36876/pdf/0?code=iWputJJBgwPmGZtWXNRMjZzj99kqWU6K9wLErJcnxxbVwmhMR6ihPJYiRV5roSGqjxx9hiuDX0vtSwqXC4JrwA==>